

RAG-Augmented Multi-Agent LLM Framework for Explainable Software Vulnerability Detection Using CWE Knowledge Bases

Richa Verma (25MCSS02) Advisor: Dr. Akshay Pandey

What we want the tool to explain

Where this fits

Many scanners only say that a piece of code is unsafe. The programmer still has to guess the kind of mistake and the fix.

This note explains the problem we are working on. Later notes will describe the files, the knowledge book, the search, and the scores. Those pieces are not built in this note.

The idea we started with

The first idea was this: a vulnerability finding is useful only if a programmer can check it. A bare yes or no cannot be checked. A named weakness, the lines involved, and a fix can be checked.

We also started from a second idea. The name of the weakness should come from a public list, not from a private label we invent. CWE is that list. If the note says CWE-89, anyone can open the CWE page and see whether our explanation matches it.

Why we chose this way

We did not start by trying to beat published detectors. Other systems already combine search, agents, and CWE catalogs. A higher score on a famous test set would not, by itself, explain a bug to a person.

We limited the study to six Java weaknesses that show up in ordinary web and enterprise code: database queries, web pages, file paths, saved objects, passwords written in source, and weak cryptography. Six is small enough to explain carefully. It is not a claim that other bugs do not matter.

Java was chosen because the public teaching suites and the industry-style suites we can later measure are in Java. Starting with many languages would have mixed the question before we knew whether the explanation itself was sound.

What we actually did

We want a tool that reads a short Java method and writes a note a person can check.

The note should say four things: whether the code is unsafe, which weakness it is, which lines matter, and how to fix it.

The weakness name comes from CWE. CWE is a public list of common software mistakes. We use the official name so two people mean the same thing.

We are studying six Java weaknesses only. They are listed in the table. They cover database queries, web pages, file paths, saved objects, passwords in code, and weak encryption.

We are not trying to replace a commercial scanner. The aim is an explanation that can be traced back to a CWE page and to real lines of code.

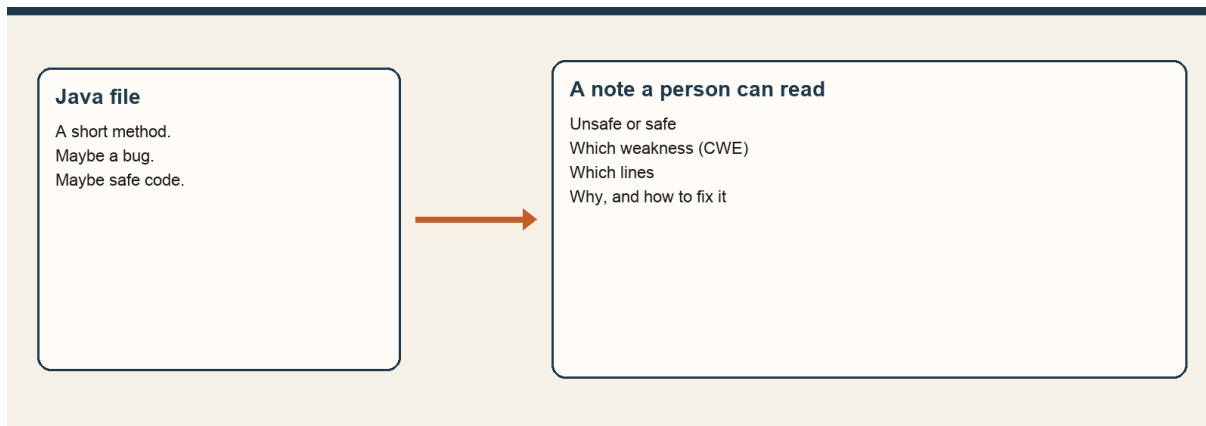
A concrete example

Suppose a method builds a database query by joining a username onto the text of the query. A yes-or-no scanner might only say "unsafe".

The note we want is longer. It should say the code is unsafe, the weakness is CWE-89 SQL injection, the risky lines are the ones that build the query, and the fix is to use a prepared query so the username is data, not part of the command.

If the same method uses a prepared query, the note should say the code is safe, and it should still name CWE-89 so a reader knows which risk was checked.

Picture



The left box is the Java we read. The right box is the note we want back.

The numbers

Weakness	Plain meaning
CWE-89	SQL injection. User text is stuck into a database query.
CWE-79	Cross-site scripting. User text is shown as part of a web page.
CWE-22	A file path can be steered outside the intended folder.
CWE-502	Loading a saved object can run attacker-controlled data.
CWE-798	A password is written directly in the code.
CWE-327	The code uses weak encryption or a weak hash.

These six are the whole scope. A different kind of bug is outside this project, and we will not relabel it to force a match.

What we concluded, and why

The next piece of work is a small set of Java examples, half unsafe and half safe, so the idea can be tried on real files.

Until those files exist, there is nothing to score. The conclusion of this note is only the question: Java in, a checkable CWE note out.

What we are not saying

This note has no test scores. It does not say the tool is better than any other scanner.